

Overview

<https://github.com/abstjohn04/RCETI-SP26>

This handoff packet details what team CSE 2.3 completed, the next steps that should be taken, and how to install and use the system.

What was completed

Implemented 4 servo design

CSE 2.3 successfully updated the system to support the new 4 servo design. Specifically, it added new variables to the math engine to allow for full movement in the x direction (yaw) and y direction (pitch), added the new 4 servo mount to the simulation's meshes, and updated the hardware controller to access the new servos.

Enhanced servo movement

After adding support for the new servos, CSE 2.3 refined their operation. The math calculations were refined to have a smaller step size (allowing for more precise movements). Boundaries were added to the movements in the controller and simulation. Additionally, the code was simplified to allow for simpler modification of servo mappings.

Current math equations used in simulation:

The total bend angle (absolute angle between bottom base and tip):

$$\theta = \sqrt{motor1^2 + motor3^2}$$

The bending plane angle (direction of the bend)

$$\phi = \arctan(motor1 / motor3)$$

The Curvature (tightness of curve dependent on length of the segment)

$$\kappa = \theta / L$$

Created Machine Learning ROS Node

The machine learning algorithm was successfully integrated with the rest of the ROS ecosystem by creating a new ROS package (node) specifically to communicate with the separate machine learning codebase. This node uses a mock camera system to play the intubation video from the ML codebase. It contains the “best” (best.py) algorithm which analyzes the camera feed and broadcast movement commands to the math engine and then to the simulation and controller.

Cleaned up existing codebase

CSE 2.3 removed old files left from previous teams, removed duplicated codefiles, and added detailed documentation to new code in the form of Doxygen comments.

Future Work Recommendations

Increased testing with physical robot

CSE 2.3 was not able to thoroughly test the controller node with the physical robot due to hardware wiring issues. While simulation testing proved successful, future teams must retest all of CSE 2.3's new features on the physical hardware to ensure validity.

Additionally, the current machine learning node uses a mock camera. It is not integrated with the physical camera. To allow the machine learning algorithm to work as intended, any hardware issues with the robot must be corrected and then the camera feed must be fed into the ROS machine learning node. A mock mouth and throat must also be used to allow the machine learning algorithms to work.

Refine mathematics and bending step size

The current mathematics for the simulation, controller and machine learning must be updated to reflect the dimensions of the actual endotracheal tube to be used. The number of segments and disks and their sizes must be updated in the robot_params.yaml file to allow for accurate calculations. Other mathematical formulas, such as inverse kinematics, should be explored.

Integrate manual override of machine learning node

Currently, the keyboard node and the ML node cannot be launched simultaneously, due to a joint publishing conflict. To allow for the ability to stop the ML node and give control back to the keyboard, this conflict must be resolved.

Unified GUI

The current implementation relies on entering terminal commands and interacting through separate gnome terminals. This is unappealing and disconnected. A GUI should be created to allow for easy navigation of the robot, allow for manual keyboard override, and display detailed information about the status of the robot, camera, and ROS system.

README.md

<https://github.com/abstjohn04/RCETI-SP26/blob/main/README.md>

RCETI

Robotic Control of Endotracheal Tube Insertion

Overview

R.C.E.T.I is a ROS 2 autonomous intubation device. The system utilizes a multi axis rigid linear actuator frame to position the device at the patient's airway, and a 4 tendon continuum robotic tube to safely navigate the throat.

This project assumes the user is controlling the robot remotely via a Raspberry Pi and is using Ubuntu 22.04 LTS.

Github Branches

RCETI is currently divided into two branches.

The "main" branch contains the most recent stable code that has been tested on the physical robot. It contains updated and more precise servo movements. It does not contain the machine learning algorithms or the updated simulation environment.

The "development" branch contains the latest code. It has not been tested on the physical robot. It contains the machine learning node and a more polished simulation environment and math engine.

Requirements

Ubuntu 22.04

ROS 2

Raspberry Pi

File Structure

The workspace is divided into specific ROS 2 packages handling different levels of the robotic stack

rceti_support: Contains the URDF, Xarco, and 3D STIL meshes defining the rigid physical frame of the robot.

rceti_continuum: The core mathematical engine. It calculates the inverse kinematics (tendon lengths).

rceti_controller: The hardware interface running on the Raspberry Pi.

rceti_keyboard: A node allowing for manual control of the four independent continuum tendons via keyboard inputs

rceti_deployment: The launch files that tie the simulation, hardware, and math nodes together. Contains robot parameters in config/robot_params.yaml

rceti_vision: The input node for the machine learning algorithms.

Installation

See INSTALL.md to install ROS 2 and setup the environment on the Ubuntu host machine.

Network Setup

See NETWORK.md to access the Raspberry Pi.

Usage

Before launching any ROS node, the user must first source the environment and build.

```
colcon build --cmake-args -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

```
source install/setup.bash
```

Afterwards, the full network can be launched.

Commands

```
colcon build --packages-select package_name - Builds a specific node
```

```
ros2 launch rceti_deployment rceti_deployment.launch.xml - Runs the whole robot system with machine learning disabled. Manual keyboard controls enabled.
```

```
ros2 launch rceti_deployment rceti_deployment.launch.xml use_ai:=true - Runs the whole robot system with machine learning enabled. Launches mock camera and vision bridge. Manual keyboard controls disabled. UNAVAILABLE IF USING MAIN BRANCH.
```

```
ros2 launch package_name package_launch_file - Launches a package's launch file (see names of launch files in codebase).
```

```
ros2 run package_name package_run_file - Runs a specific eligible file in a node (see names of files in codebase).
```

RViz simulation tool

This tool launches with the rceti_continuum node. It fetches the modeled meshes from rceti_support and simulates them according to the urdf files (See individual nodes' .urdf and .xacro files for more details on what is launched and created).

To disable the Transform frames (all the green, red and blue lines) in the simulation, uncheck the "TF" button in the left settings screen.

Moving the robot

If you are using the machine learning node with the mock camera, it will automatically move for you!

To manually move the robot, the rceti_keyboard node must be launched. This will launch a separate terminal which outputs coordinates.

To adjust the position of the servo mount, use A and D to move left and right, and W and S to move up and down.

Use L and P to adjust the angle of the servo mount.

To move the tube, use the left and right arrow keys to adjust the Y coordinate (pitch) and the up and down arrow keys to adjust the X coordinate (yaw).

License & Acknowledgements

This project is licensed under the Apache License 2.0.

Original Work: This repository is a derivative work based on the original RCETI robotics project. For a list of original authors, please visit the contributors page of the original repository at <https://github.com/bturner86239/RCETI/graphs/contributors>.

Modifications: In 2026, CSE 2.3 modified the source code, hardware architecture, and simulation environment to implement an independent 4-servo perpendicular continuum drive.

INSTALL.md

<https://github.com/abstjohn04/RCETI-SP26/blob/main/docs/INSTALL.md>

RCETI System Setup & Installation

Setup Ubuntu Host System

First, we need to ensure the Ubuntu host machine is up to date and has the basic tools required to download the robotics software.

Open a new Terminal window (Ctrl + Alt + T).

Enter the following command and press enter:

```
sudo apt update && sudo apt upgrade -y
```

(Note: If prompted with "A new version of Ubuntu is available. Would you like to upgrade?", select "Do Not Upgrade". The robot specifically requires Ubuntu 22.04)

Restart the Ubuntu host machine

Install ROS 2

RCETI runs on ROS 2 Humble. Run the following commands one by one in your terminal to install the core robotic libraies.

Add the ROS 2 Repository

```
sudo apt install software-properties-common
```

```
sudo add-apt-repository universe
```

```
sudo apt update && sudo apt install curl -y
```

```
export ROS_APT_SOURCE_VERSION=$(curl -s  
https://api.github.com/repos/ros-infrastructure/ros-apt-source/releases/latest | grep -F  
"tag_name" | awk -F\" '{print $4}')
```

Install the Core Software

```
curl -L -o /tmp/ros2-apt-source.deb
```

```
"https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE  
_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.${ /etc/os-release && echo  
${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"
```

```
sudo dpkg -i /tmp/ros2-apt-source.deb
```

```
sudo apt update && sudo apt install ros-humble-desktop -y && sudo apt install  
ros-humble-moveit -y && sudo apt install python3-colcon-common-extensions -y && sudo apt  
install python3-rosdep2 -y
```

Restart the Ubuntu host machine.

Set up the ROS environment

Next we will setup the ROS environment. Enter the following commands in your terminal one by one.

Source the setup files:

```
source /opt/ros/humble/setup.bash
```

If sourcing the files works, make it run automatically

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

Close the current terminal and open a new one (Ctrl + Alt + T)

Create a new folder named "rceti_ws" with a subfolder named "src" in your home directory to store the code:

```
cd && mkdir -p rceti_ws/src
```

Download & Build the RCETI Code

Now that the robotics software is installed, we will download the RCETI code and the Machine Learning code. Once again, follow the steps and enter the commands in your terminal.

Install git:

```
sudo apt update && sudo apt install git -y
```

Clone the RCETI repo to rceti/src (make sure to include the dot at the end of the command!):

```
cd ~/rceti/src && git clone https://github.com/abstjohn04/RCETI-SP26 .
```

Clone the Machine Learning repo to a new folder called ML (or any folder you wish):

```
cd && mkdir ML && cd ML && git clone https://github.com/github4bme/EndotrachealTubeModel .
```

(Note: The machine learning repo is massive. It may take up to a hour to fully download.)

If you wish to use the mock machine learning camera, you must change the file path for "video_path" in rceti/src/rceti_vision/rceti_vision/mock_camera.py to match the location of your cloned machine learning repo.

Enter the rceti_ws/ directory and install dependencies:

```
cd ~/rceti_ws
```

```
rosdep update
```

```
rosdep install -i --from-path src --rosdistro humble -y
```

```
colcon build
```

Restart the Ubuntu host machine (CRUCIAL!)

Validating the installation

To test the RCETI robot, enter the following command:

```
cd ~/rceti_ws/ && source install/setup.bash && ros2 launch rceti_deployment  
rceti_deployment.launch.xml
```

If the installation and launch was successful, three things will open on your screen:

The 3D Simulation (RViz): The 3D live model of the robot.

The Data Terminal: A window displaying the real time math calculations and the coordinates.

The System Log: The original terminal where you typed the command will begin streaming the system health status.

To close the application, navigate to the System Log terminal window and press "Ctrl + C".

Close the other windows manually if needed.

See NETWORK.md for instructions on how to access the Raspberry Pi.

See the "Usage" section in README.md on how to use the system.

NETWORK.md

<https://github.com/abstjohn04/RCETI-SP26/blob/main/docs/NETWORK.md>

RCETI Network Setup
Setting Up Static IP on Ubuntu Host
Open Settings > Network.

Click on Wired Connection (the Ethernet interface).

Click Settings (⚙️).

Go to the IPv4 tab.

Select Manual and set:

Address: 192.168.2.1
Netmask: 255.255.255.0
Gateway: Leave blank
Click Apply and disconnect/reconnect Ethernet.

Setting Up ROS to use static IP

Run these commands in your Ubuntu 22.04 terminal to bind ROS 2 traffic to the Ethernet interface. This will allow communication with the Raspberry Pi.

```
echo "export ROS_DOMAIN_ID=7" >> ~/.bashrc
echo "export ROS_IP=$(hostname -I | awk '{print $1}')" >> ~/.bashrc
echo "export ROS_HOSTNAME=$(hostname -I | awk '{print $1}')" >> ~/.bashrc
source ~/.bashrc \
```